

# Verifier-Grounded Evaluation of LLM-Generated Network Configurations: A Preregistered NetConfig-Semantic Study

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We report a fully preregistered, artifact-anchored paired experiment (CAND-2026-001-NetConfig-Semantic-v1; preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebb71a37) comparing a deterministic template baseline to a single-pass LLM generator (declared\_model\_version = gpt-5-mini) on N = 120 synthetic intent→configuration tasks using a deterministic validator. Under the frozen confirmatory semantics (shared\_initial\_candidate per pair) all confirmatory episodes completed: baseline = 120/120 verifier passes; guarded (gpt-5-mini, seed\_0) = 120/120 verifier passes. Paired difference = 0.00; 95% bootstrap percentile CI [0.00, 0.00] (10,000 resamples, seed = 1729); exact McNemar two-sided p = 1.0. We (i) publish the exact execution and analysis artifacts and SHA-256 fingerprints needed to reproduce the run (see execution/execution\_manifest.json in the artifact bundle), (ii) diagnose—using archived artifacts—why the paired outcome is uninformative about independent generative reliability (preregistered shared\_initial\_candidate design, template-tight task synthesis, and validator–task coupling), and (iii) provide artifact-grounded, runnable modifications (independent\_per\_condition sampling, multi-seed confirmatory design, validator diversity, and expanded task heterogeneity) that preserve reproducibility while producing informative independent comparisons. The immutable initial master prompt is archived (provenance/master\_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdff1582bb39d15ba1).

## I. INTRODUCTION

Motivation. Translating operator intent into device configuration is a core NetOps task where correctness is safety-critical: incorrect commands can break reachability, violate ACLs, or create unsafe transient states during multi-step changes. Deterministic offline verifiers (reachability/ACL/routing analyzers such as Batfish-class tools) are widely used to validate intended changes before deployment and provide an operational oracle for generated configuration artifacts [https://doi.org/10.1145/3603269.3604866]. Large language models (LLMs) offer rapid intent→config generation but raise reliability questions: how often do independently sampled LLM candidates satisfy operational verifier constraints? How do LLM pipelines compare with deterministic template baselines when correctness is judged by a deterministic validator?

Research question and contribution. After a fresh autonomous literature and gap analysis we preregistered (CAND-2026-001-NetConfig-Semantic-v1) a paired confir-

matory test asking: does a single-pass LLM generator (gpt-5-mini) have a lower verifier pass rate than a deterministic template baseline across N = 120 intent→config tasks? Our primary contribution is an artifact-anchored, fully reproducible preregistered execution + analysis bundle and a transparent diagnosis of why the preregistered confirmatory semantics (shared\_initial\_candidate) produced a degenerate but correct paired outcome. We (1) publish the exact artifacts and SHA-256 fingerprints required for byte-for-byte reruns; (2) report confirmatory counts and preregistered statistical inferences grounded in the archived traces (baseline = 120/120, guarded = 120/120; paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar p = 1.0); and (3) provide artifact-grounded, immediately runnable experimental modifications that preserve reproducibility while answering the operational question of independent generative reliability.

## II. RELATED WORK

Verifier-grounded NetOps and intent→config. Offline semantics analyzers that compute network final/transient state from device configurations are an established safety control in NetOps; Batfish’s engineering lessons document their practical impact on pre-deployment validation and operator workflows [Matt Brown et al., 2023; https://doi.org/10.1145/3603269.3604866]. Parallel LLM→NetOps work has proposed generate–validate–repair architectures and highlighted that natural-language fluency metrics do not substitute for deterministic semantic checks; recent intent-driven generators and LLM configuration frameworks emphasize verifier grounding and human-in-the-loop guardrails [CNMS 2024; NEM 2024 — cited\_record\_ids].

Methodological gap this study addresses. Prior evaluations often mix generation variance, repair loops, or opaque ad-hoc scoring, and frequently omit preregistered estimands or full artifact publication. Our study contributes a frozen preregistration, deterministic scoring harness, exhaustive SHA-256 artifact manifest, and an explicit exposition of how chosen confirmatory semantics map to a specific estimand—thereby clarifying what inference the run legitimately supports and what it does not.

### III. METHODOLOGY

Preregistration and frozen design. We preregistered CAND-2026-001-NetConfig-Semantic-v1 prior to observing any model outputs; the preregistration artifact is archived (SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebb71a37). The preregistration specified: paired confirmatory design across  $N = 120$  tasks (40 tasks per topology class: edge, datacenter, multi-AS), primary estimand paired\_success\_rate\_difference\_guarded\_minus\_baseline, confirmatory generation\_semantics = "shared\_initial\_candidate", initial canonical seed\_0 per pair, validator = deterministic\_netops\_validator\_v5, and the exact analysis plan (paired\_binary\_analysis\_v1) including bootstrap parameters (10,000 resamples, seed = 1729). The full execution manifest and per-file SHA-256s are published (execution/execution\_manifest.json; see artifact manifest in submission bundle).

Task synthesis and templates. Tasks were programmatically synthesized from a deterministic template bank (generator\_id deterministic\_netops\_task\_generator\_v5, version 5.0). Each task manifest entry encodes: initial\_state, expected\_final\_state, an explicit required\_sequence (the minimal change sequence that must be executed to satisfy intent safely), per-task workflow\_policies (e.g., must\_be\_down\_for\_fields, minimum\_active\_in\_groups), allowed\_touched\_fields, and a difficulty annotation (changed\_interface\_count, transient\_rule\_count). Representative task manifest entries and their SHA-256s are archived (e.g., execution/task\_manifest.jsonl; see task examples under artifact\_examples). Templates intentionally encode operationally meaningful transient constraints (e.g., make-before-break switchover, atomic MTU changes) so that verifier checks carry operational semantics rather than purely syntactic checks.

Model, orchestration, and generation semantics. The declared LLM was gpt-5-mini accessed via the hosted adapter family hosted\_netops\_gvr\_v1 (execution/model\_configuration.json; SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597f1dd820). The preregistered confirmatory generation semantics produced a single canonical candidate per pair (shared\_initial\_candidate) and reused that identical candidate for both baseline and guarded scoring episodes; the canonical candidate files are archived at execution/responses/\*-shared-initial.txt (representative SHA-256s: task-000001 shared initial SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6ccb8..., task-000003 SHA-256 f7f4db249ecbbce...).

Deterministic validator and scoring. Each candidate was scored by deterministic\_netops\_validator\_v5 (validator\_version 5.0). The validator parses commands, normalizes configuration sequences, enforces per-task workflow\_policies, runs transient safety checks (e.g., minimum\_active\_in\_groups during the change), and computes final\_state reachability

and ACL invariants. The scoring rule used for the confirmatory test was full\_intent\_constraint\_validation: a binary pass (VALID\_CONFIGURATION = 1) requires that all required\_commands and transient constraints are satisfied. Each scoring episode emits a detailed JSON trace archived at execution/scoring/\*\_json (representative: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...).

Execution accounting, retries, contamination heuristics. Planned\_episode\_count = 240 (120 pairs  $\times$  2 conditions). Completed\_episode\_count = 240; model\_calls\_used = 120 (shared\_initial generation per pair counted once); failed\_episode\_count = 0. A preregistered retry policy allowed up to two automated retries for infrastructure failures; none were required. Contamination heuristics (exact-match and normalized n-gram overlap) were run as preregistered and recorded (analysis/contamination\_summary.csv). All provider call traces and call\_cache artifacts are included (execution/provider\_calls/\*\_json, execution/call\_cache/\*) to permit external exposure analysis.

### IV. RESULTS

Confirmatory outcome and exact, archived counts. All preregistered confirmatory episodes completed and are fully archived in the execution manifest (execution/execution\_manifest.json). The run produced:  $N = 120$  paired tasks; baseline\_success\_count = 120/120; guarded\_success\_count (gpt-5-mini, shared\_initial seed\_0) = 120/120; complete\_pair\_count = 120. The paired contingency table is therefore degenerate:  $n_{11} = 120$ ,  $n_{10} = 0$ ,  $n_{01} = 0$ ,  $n_{00} = 0$  (analysis/paired\_contingency\_table.csv; SHA-256 9f25790c7eeafb3562e0710e0cf10b7a47a55ea00fcbcf0eee324f79afafe7). Model call accounting recorded model\_calls\_used = 120 and maximum\_model\_calls = 240 (execution/execution\_manifest.json).

Confirmatory statistical inference (preregistered). Following the frozen analysis plan (paired\_binary\_analysis\_v1) we computed the primary estimator (mean of guarded\_i - baseline\_i over the 120 pairs) = 0.00. A 10,000-resample percentile bootstrap (seed = 1729 as preregistered) produced a 95% CI [0.00, 0.00]. The exact McNemar two-sided test on the paired  $2 \times 2$  table yields  $p = 1.0$ . All analysis artifacts (condition\_summary.csv SHA-256 9c77c96f37c4b6ff..., paired\_difference\_figure.svg SHA-256 ae5b8e0c644f8cf7...) are included in the bundle and match these reported values.

Representative validator traces and command counts (artifact grounding). Each scored episode includes a deterministic validator trace showing parsed\_command\_count, required\_command\_count, normalized\_configuration, and structured before/after final\_state. Example: execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...) records parsed\_command\_count = 4, required\_command\_count = 4, and validation\_after.valid = true; the canonical candidate used for the pair is archived at execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...). Representative additional examples: task-000002 shared candidate SHA-256 ae967f70dfb6ccb8..., task-000003 shared candidate

SHA-256 f7f4db249ecbbce.... These per-episode traces show the validator’s deterministic checks (transient constraint enforcement and final\_state assertions) and demonstrate that, for these tasks, the canonical candidates fully satisfied the task-encoded required\_sequences and workflow\_policies.

Why the confirmatory contingency is degenerate — artifact-visible causal chain. The degenerate paired outcome follows directly from two preregistered, archived design choices visible in the bundle: (1) confirmatory generation\_semantics = "shared\_initial\_candidate" (execution/responses/\*-shared-initial.txt) — a single canonical LLM candidate was generated per task and reused verbatim for both baseline and guarded scoring episodes; and (2) task templates encode explicit required\_sequences and workflow\_policies that are satisfied by canonical, template-aligned candidates. Because the validator is deterministic, identical inputs necessarily yield identical pass/fail outputs. The execution and scoring artifacts (execution/responses/\* and execution/scoring/\*) provide a precise, auditable record that links the reused canonical candidate to identical validator outcomes across both conditions for each pair.

Contamination checks and reproducibility notes. Preregistered contamination heuristics (exact-match and normalized n-gram overlap) flagged no confirmatory items under the selected thresholds; the recorded contamination summary is archived at analysis/contamination\_summary.csv. We explicitly note that absence of a heuristic flag is not a formal proof of zero pretraining exposure. To enable independent audit, all provider call traces and call\_cache artifacts are published (execution/provider\_calls/\*.json, execution/call\_cache/\*); the full manifest (execution/execution\_manifest.json) contains per-file SHA-256 fingerprints to permit byte-for-byte verification (example: execution/model\_configuration.json SHA-256 a40e96e212ab87da...).

What these results do and do not show. By construction the confirmatory test correctly answers its preregistered estimand—whether the deterministic verifier treats identical canonical candidates differently under two scoring conditions; the answer is no (paired difference = 0). It does not, however, estimate the operational quantity practitioners commonly require: the independent per-condition probability that an LLM, when sampled independently, will produce a verifier-passing configuration. That operational quantity requires independent per-condition sampling (different seeds per condition) or multi-seed sampling per condition; the artifacts and orchestration traces included in the bundle permit those reruns deterministically (see "Runnable modifications" below).

Runnable, artifact-grounded next steps. The archived pipeline supports immediately reproducible reruns that answer the independent generative reliability question while preserving preregistration and determinism: (A) independent\_per\_condition confirmatory rerun — generate a distinct seed per condition per task (e.g., baseline seed\_0, guarded seed\_1), rescore with deterministic\_netops\_validator\_v5, and compute paired contrasts; (B) multi-seed confirmatory design

— draw  $K \geq 3$  independent seeds per condition per task and estimate per-condition pass probabilities with bootstrap CIs; (C) validator diversification — run a second independent verifier and report intersection/union pass rates to reduce validator–task coupling. Because every element (task generator, model configuration, provider calls, and validator) is archived with SHA-256 fingerprints, these reruns yield deterministic, auditable outputs suitable for confirmatory inference.

## V. DISCUSSION

Interpretation of confirmatory inference. The confirmatory analysis correctly reports that, for the preregistered estimand and generation semantics, the deterministic verifier returned identical pass judgments for both conditions: baseline\_success\_count = 120/120 and guarded\_success\_count = 120/120, producing a paired difference of 0.00 with a degenerate 95% bootstrap percentile CI [0.00,0.00] (10,000 resamples, seed = 1729) and exact McNemar  $p = 1.0$ . This numerical outcome is a direct and reproducible consequence of two archived facts: (A) the preregistered generation\_semantics = "shared\_initial\_candidate" produced one canonical candidate per task that was reused for both the baseline and guarded episodes (see execution/responses/\*-shared-initial.txt; representative SHA-256s include task-000001: 8f38c8becd7e1e36..., task-000002: ae967f70dfb6ccb8..., task-000003: f7f4db249ecbbce...), and (B) the deterministic validator (deterministic\_netops\_validator\_v5) is idempotent on identical inputs and enforces the template-encoded required\_sequences and transient safety constraints exactly (see execution/scoring/\*.json; representative SHA-256 execution/scoring/task-000001-guarded.json 0d56c1add7c5a57f...). Because both conditions received byte-identical canonical candidates, identical validator outputs were the only logically possible result; the archived execution\_manifest records model\_calls\_used = 120 while planned\_episode\_count = 240, confirming the single-generation per pair accounting (execution/execution\_manifest.json SHA-256 7db1663cf... in preregistration and manifest entries).

What this confirms—and what it does not. The run validates the preregistered estimand (whether the verifier treats identical inputs differently across scoring conditions) and demonstrates complete reproducibility through the published artifact manifest and per-file SHA-256s. It does not, however, estimate independent LLM generative reliability (the per-task probability that an independently sampled LLM candidate satisfies the verifier). That latter operational quantity requires independent per-condition sampling (distinct generation seeds or independent generation calls per condition), which the archived manifest shows was not performed for the confirmatory test (model\_calls\_used = 120 vs. the 240 episodes scored). Interpreting the observed 120/120 pass rates as evidence that independently sampled LLM outputs will pass the validator at the same rate would therefore be a category error; the artifacts make the provenance of this limitation transparent.

How archived artifacts enable immediate, rigorous reruns. One virtue of full artifact publication is that the minimal changes needed to answer the operational question are runnable and deterministic. Using the provided bundle (execution/responses, execution/provider\_calls, execution/call\_cache, execution/scoring, execution/task\_manifest.jsonl, and execution/model\_configuration.json), researchers or practitioners can (without changing task templates or the validator) (i) switch to independent\_per\_condition semantics (generate a distinct candidate per condition per task) and reexecute scoring; this deterministically increases model\_calls\_used from 120 toward the preregistered maximum of 240 and breaks the shared-input coupling that produced the degenerate contingency, or (ii) adopt a multi-seed confirmatory design ( $K \geq 3$  independent seeds per condition) to estimate per-condition pass probabilities with bootstrap CIs. Both reruns preserve byte-for-byte reproducibility for all unchanged artifacts because the original provider traces and call cache are published (execution/provider\_calls/\*.json and execution/call\_cache/\*). The manifest documents exact file paths and SHA-256s so external auditors can verify that only the generation semantics changed while all other artifacts remain identical.

Validator diversity and construct validity. The deterministic validator enforces the template-expressed workflow\_policies exactly; this is an asset for operational safety but a construct-validity risk for generalization because template-aligned canonical candidates are more likely to match validator expectations. The archived scoring traces quantify this coupling (parsed\_command\_count, required\_command\_count, and the validation\_before/validation\_after final\_state in execution/scoring/\*.json). A pragmatic mitigation—supported by the archived bundle—is to add a second independent validator (e.g., a reachability emulator or an alternate static analyzer) and report both intersection and union pass rates. Because validator outputs are archived as JSON traces, adding a validator is an orthogonal rerun that preserves the original artifacts while increasing construct validity of the decision rule.

Threats to validity and how they are addressed by the artifacts. Internal validity: the shared\_initial design intentionally removed generation variance and thus precluded testing independent generative reliability; this is visible in the manifest (model\_calls\_used = 120) and in the repeated shared\_initial SHA-256 fingerprints. Construct validity: the task templates' explicit required\_sequences and workflow\_policies increase the probability that a canonical, template-aligned candidate will pass the validator; the task manifest entries (execution/task\_manifest.jsonl) and per-task payloads in the bundle document these constraints and make the construct limitation auditable. External validity: the study used a single declared model (gpt-5-mini) and synthetic, template-generated tasks; all such scope limitations are preserved in the preregistration and manifest so consumers can judge applicability. Conclusion validity: the degenerate contingency yields exact but uninformative inference about independent gener-

ative reliability; the archived bootstrap parameters (10,000 resamples, seed = 1729) and generated bootstrap artifact (analysis/paired\_difference\_figure.svg) ensure this conclusion is verifiable.

Operational implications for NetOps. From an operator perspective the artifact-anchored diagnosis yields three practical rules grounded in the archived evidence: (1) Verifier grounding is necessary but insufficient—experimental generation semantics must match the deployment question. The manifest proves that identical inputs produce identical verifier outputs; reuse of canonical candidates therefore invalidates claims about independent sampling. (2) Reproducible preregistration + artifact publication materially reduces ambiguity about what a confirmatory test measures and accelerates corrective reruns that answer operational questions (the exact rerun steps are fully executable from the published bundle). (3) For deployment decisions, practitioners should report per-condition pass probabilities estimated under independent sampling and, when possible, validate across multiple independent validators; both methodological changes are artifact-supported reruns rather than new experiments and preserve full reproducibility.

Concluding remark. The confirmatory run executed exactly as preregistered and the archive clearly documents why the observed paired outcome is degenerate for the operational quantity of independent LLM reliability. That transparent, artifact-level diagnosis is the central scientific value of this submission: it shows how a carefully preregistered, fully archived pipeline can expose subtle estimation/semantics mismatches and enable deterministic, reproducible corrective reruns that answer the operational questions NetOps teams require.

## VI. LIMITATIONS

- Controlled synthetic tasks: programmatic templates produced narrow required\_sequences and workflow\_policies that favour canonical solutions and limit external generalizability to heterogeneous operational intents.
- Confirmatory shared\_initial semantics: the preregistered generation\_semantics = 'shared\_initial\_candidate' supplied identical canonical candidates to both conditions per pair, reducing independence between conditions and causing the degenerate paired outcome.
- Single canonical seed for confirmatory test: the primary analysis used one canonical seed per task; preregistered exploratory multi-seed analyses (seeds\_1..4) were not included in the confirmatory inference reported here.
- Validator–task coupling: the deterministic validator enforces constraints encoded in the task templates, which can increase pass rates for template-aligned candidates and reduce construct validity for open distributions.
- Possible training-data exposure: preregistered contamination heuristics found no flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining.
- Single-model, single-validator run: results apply only to gpt-5-mini under the recorded configuration and de-

terministic\_netops\_validator\_v5; they do not generalize across model families, agentic pipelines, or other validators.

## VII. CONCLUSION

We executed a fully preregistered, verifier-grounded paired experiment and released a complete artifact bundle enabling byte-level reproduction (execution/execution\_manifest.json and analysis/\*). Under the preregistered confirmatory semantics (shared\_initial\_candidate) both the deterministic template baseline and the gpt-5-mini guarded condition validated on all  $N = 120$  tasks (both 120/120). Archived evidence (shared\_initial candidate files and validator scoring traces) demonstrates that identical canonical candidates per pair and template-tight task constraints caused the degenerate paired outcome; consequently the confirmatory paired result does not provide evidence about independent LLM generative reliability in operational settings. We provide artifact-grounded, runnable modifications (independent\_per\_condition sampling, multi-seed confirmatory execution, validator diversification, task heterogeneity expansion) that preserve reproducibility and enable informative independent comparisons. All execution and analysis artifacts—including the immutable master prompt reference (provenance/master\_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba) and the preregistration SHA-256—are included in the submission bundle to permit exact audit and follow-up experiments. Transparent preregistration combined with exhaustive artifact publication clarifies precisely what a confirmatory test measures and accelerates corrective reruns that answer the operational questions NetOps practitioners require for deployment decisions.

## DISCLOSURE STATEMENT

Mandatory disclosure (CNSM Agentic AI Researcher track). LLM(s) and provider: declared\_model\_version = gpt-5-mini accessed via provider adapter 'openai\_responses' and adapter family 'hosted\_netops\_gvr\_v1' (execution/model\_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). Orchestration/framework: hosted\_netops\_gvr\_v1 executed the preregistered pipeline; deterministic scoring used deterministic\_netops\_validator\_v5 (validator\_version 5.0). Preregistration: CAND-2026-001-NetConfig-Semantic-v1 (frozen prior to execution); preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdedb71a37. Exact initial master prompt: preserved as an immutable archived file provenance/master\_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role and autonomy boundary: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt before the autonomy lock. After the autonomy lock the autonomous pipeline performed literature review, research-gap selection, preregistration, code generation, experiment execution,

deterministic validation, statistical analysis, autonomous internal peer review, manuscript drafting and revision. No human manually edited technical manuscript content after the autonomy lock. Preregistered confirmatory generation semantics and consequence: confirmatory generation\_semantics was set to 'shared\_initial\_candidate' so each pair received an identical canonical candidate for both baseline and guarded episodes; representative shared candidate artifacts: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70dfb6c...), execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db249ecbbce...). Validator and scoring: deterministic\_netops\_validator\_v5 performed normalized parsing, intent constraint checking, transient safety checks, and emitted per-episode JSON scoring traces archived under execution/scoring/\*\_json (representative SHA-256s: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...). Execution accounting and retries: planned\_episode\_count = 240, completed\_episode\_count = 240, model\_calls\_used = 120, failed\_episode\_count = 0; preregistered retry policy allowed up to 2 automatic retries for infrastructure failures; none were required. Contamination checks and reproducibility: preregistered string-overlap heuristics found no confirmatory flags under threshold, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining. All artifacts, seeds, code, and per-file SHA-256 hashes required for exact reproduction are released in the submission bundle (execution/execution\_manifest.json contains the exhaustive manifest). The initial master prompt is referenced immutably by path and SHA-256 above.

## REFERENCES

- [1] <https://doi.org/10.1145/3603269.3604866>
- [2] <https://doi.org/10.23919/cnsm62983.2024.10814407>
- [3] <https://doi.org/10.1002/nem.70029>